NanoWasm Specification

The SpecTec Team

CONTENTS

1 Abstract Syntax **3**

2 Validation **5**
2.1 nop 5
2.2 drop 5
2.3 select 5
2.4 const 5
2.5 local.get 5
2.6 local.set 6
2.7 global.get 6
2.8 global.set 6

3 Execution **7**
3.1 nop 7
3.2 drop 7
3.3 select 7
3.4 local.get x 8
3.5 local.set x 8
3.6 global.get x 8
3.7 global.set x 8

4 Binary Format **9**

NanoWasm is a small language with simple types and instructions.

CHAPTER

ONE

ABSTRACT SYNTAX

The *abstract syntax* of types is as follows:

$$
\begin{array}{rcl}
mut & ::= & \mathsf{mut} \\
valtype & ::= & \mathsf{i32} \mid \mathsf{i64} \mid \mathsf{f32} \mid \mathsf{f64} \\
functype & ::= & valtype^* \to valtype^* \\
globaltype & ::= & mut^? \; valtype
\end{array}
$$

Instructions take the following form:

$$
\begin{array}{rcl}
const & ::= & 0 \mid 1 \mid 2 \mid \dots \\
instr & ::= & \mathsf{nop} \\
& \mid & \mathsf{drop} \\
& \mid & \mathsf{select} \\
& \mid & valtype.\mathsf{const}\ const \\
& \mid & \mathsf{local.get}\ localidx \\
& \mid & \mathsf{local.set}\ localidx \\
& \mid & \mathsf{global.get}\ globalidx \\
& \mid & \mathsf{global.set}\ globalidx
\end{array}
$$

The instruction nop does nothing, drop removes an operand from the stack, select picks one of two operands depending on a condition value. The instruction t.const c pushed the constant c to the stack. The remaining instructions access local and global variables.

CHAPTER

TWO

VALIDATION

NanoWasm instructions are *type-checked* under a context that assigns types to indices:

$$context \quad ::= \quad \{\mathsf{globals}\ globaltype^*, \mathsf{locals}\ valtype^*\}$$

2.1 nop

nop is valid with $\epsilon \ \rightarrow\ \epsilon$.

$$\frac{}{C \vdash \mathsf{nop} : \epsilon \rightarrow \epsilon}$$

2.2 drop

drop is valid with $t \ \rightarrow\ \epsilon$.

$$\frac{}{C \vdash \mathsf{drop} : t \rightarrow \epsilon}$$

2.3 select

select is valid with $t\ t\ \mathsf{i32} \ \rightarrow\ t$.

$$\frac{}{C \vdash \mathsf{select} : t\ t\ \mathsf{i32} \rightarrow t}$$

2.4 const

$(t.\mathsf{const}\ c)$ is valid with $\epsilon \ \rightarrow\ t$.

$$\frac{}{C \vdash t.\mathsf{const}\ c : \epsilon \rightarrow t}$$

2.5 local.get

$(\mathsf{local.get}\ x)$ is valid with $\epsilon \ \rightarrow\ t$ if:

- $C.\mathsf{locals}[x]$ exists.
- $C.\mathsf{locals}[x]$ is of the form t.

$$\frac{C.\mathsf{locals}[x] = t}{C \vdash \mathsf{local.get}\ x : \epsilon \rightarrow t}$$

2.6 local.set

$(\mathsf{local.set}\ x)$ is valid with $t\ \rightarrow\ \epsilon$ if:

- $C.\mathsf{locals}[x]$ exists.
- $C.\mathsf{locals}[x]$ is of the form t.

$$\frac{C.\mathsf{locals}[x] = t}{C \vdash \mathsf{local.set}\ x : t \rightarrow \epsilon}$$

2.7 global.get

$(\mathsf{global.get}\ x)$ is valid with $\epsilon\ \rightarrow\ t$ if:

- $C.\mathsf{globals}[x]$ exists.
- $C.\mathsf{globals}[x]$ is of the form $(\mathsf{mut}^?\ t)$.

$$\frac{C.\mathsf{globals}[x] = \mathsf{mut}^?\ t}{C \vdash \mathsf{global.get}\ x : \epsilon \rightarrow t}$$

2.8 global.set

$(\mathsf{global.get}\ x)$ is valid with $t\ \rightarrow\ \epsilon$ if:

- $C.\mathsf{globals}[x]$ exists.
- $C.\mathsf{globals}[x]$ is of the form $(\mathsf{mut}\ t)$.

$$\frac{C.\mathsf{globals}[x] = \mathsf{mut}\ t}{C \vdash \mathsf{global.get}\ x : t \rightarrow \epsilon}$$

CHAPTER

THREE

EXECUTION

NanoWasm execution requires a suitable definition of state and configuration:

$$
\begin{array}{rcl}
addr & ::= & 0 \mid 1 \mid 2 \mid \ldots \\
moduleinst & ::= & \{\mathsf{globals}\ addr^*\} \\
val & ::= & \mathsf{const}\ valtype\ const \\
store & ::= & \{\mathsf{globals}\ val^*\} \\
frame & ::= & \{\mathsf{locals}\ val^*, \mathsf{module}\ moduleinst\} \\
state & ::= & store; frame \\
config & ::= & state; instr^*
\end{array}
$$

We define the following auxiliary functions for accessing and updating the state:

$$
\begin{array}{lcl}
\mathrm{local}((s; f), x) & = & f.\mathsf{locals}[x] \\
\mathrm{global}((s; f), x) & = & s.\mathsf{globals}[f.\mathsf{module}.\mathsf{globals}[x]] \\
\mathrm{update}_{local}((s; f), x, v) & = & s; f[.\mathsf{locals}[x] = v] \\
\mathrm{update}_{global}((s; f), x, v) & = & s[.\mathsf{globals}[f.\mathsf{module}.\mathsf{globals}[x]] = v]; f
\end{array}
$$

With that, execution is defined as follows:

3.1 nop

1. Do nothing.

$$\mathsf{nop} \quad \hookrightarrow \quad \epsilon$$

3.2 drop

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.

$$val\ \mathsf{drop} \quad \hookrightarrow \quad \epsilon$$

3.3 select

1. Assert: Due to validation, a value of valtype i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ c)$ from the stack.
3. Assert: Due to validation, a value is on the top of the stack.
4. Pop the value val_2 from the stack.
5. Assert: Due to validation, a value is on the top of the stack.
6. Pop the value val_1 from the stack.

7. If $c \neq 0$, then:
   a. Push the value val_1 to the stack.
8. Else:
   a. Push the value val_2 to the stack.

$$\begin{array}{lcll} val_1\ val_2\ (\mathsf{i32.const}\ c)\ \mathsf{select} & \hookrightarrow & val_1 & \text{if } c \neq 0 \\ val_1\ val_2\ (\mathsf{i32.const}\ c)\ \mathsf{select} & \hookrightarrow & val_2 & \text{otherwise} \end{array}$$

3.4 local.get x

1. Let z be the current state.
2. Let val be $\mathrm{local}(z, x)$.
3. Push the value val to the stack.

$$z; (\mathsf{local.get}\ x) \quad \hookrightarrow \quad z; val \quad \text{if } val = \mathrm{local}(z, x)$$

3.5 local.set x

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.

$$z; val\ (\mathsf{local.set}\ x) \quad \hookrightarrow \quad z'; \epsilon \quad \text{if } z' = \mathrm{update}_{local}(z, x, val)$$

3.6 global.get x

1. Let z be the current state.
2. Let val be $\mathrm{global}(z, x)$.
3. Push the value val to the stack.

$$z; (\mathsf{global.get}\ x) \quad \hookrightarrow \quad z; val \quad \text{if } val = \mathrm{global}(z, x)$$

3.7 global.set x

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.

$$z; val\ (\mathsf{global.set}\ x) \quad \hookrightarrow \quad z'; \epsilon \quad \text{if } z' = \mathrm{update}_{global}(z, x, val)$$

CHAPTER

FOUR

BINARY FORMAT

The following grammars define the binary representation of NanoWasm programs.

First, constants are represented in LEB format:

$$
\begin{array}{rcllll}
\texttt{byte} & ::= & \texttt{0x00} \mid \ldots \mid \texttt{0xFF} & & & \\
\texttt{u}(N) & ::= & n\texttt{:byte} & \Rightarrow & n & \text{if } n < 2^7 \wedge n < 2^N \\
 & \mid & n\texttt{:byte}\ m\texttt{:u}(N-7) & \Rightarrow & 2^7 \cdot m + (n - 2^7) & \text{if } n \geq 2^7 \wedge N > 7 \\
\texttt{u32} & ::= & \texttt{u}(32) & & & \\
\texttt{u64} & ::= & \texttt{u}(64) & & & \\
\texttt{f}(N) & ::= & b^*\texttt{:byte}^{N/8} & \Rightarrow & \mathrm{float}(N, b^*) & \\
\texttt{f32} & ::= & \texttt{f}(32) & & & \\
\texttt{f64} & ::= & \texttt{f}(64) & & &
\end{array}
$$

Types are encoded as follows:

$$
\begin{array}{rcllll}
\texttt{valtype} & ::= & \texttt{0x7F} & \Rightarrow & \mathsf{i32} \\
 & \mid & \texttt{0x7E} & \Rightarrow & \mathsf{i64} \\
 & \mid & \texttt{0x7D} & \Rightarrow & \mathsf{f32} \\
 & \mid & \texttt{0x7C} & \Rightarrow & \mathsf{f64} \\
\texttt{mut} & ::= & \texttt{0x00} & \Rightarrow & \epsilon \\
 & \mid & \texttt{0x01} & \Rightarrow & \mathsf{mut} \\
\texttt{globaltype} & ::= & t\texttt{:valtype}\ mut\texttt{:mut} & \Rightarrow & mut\ t \\
\texttt{resulttype} & ::= & n\texttt{:u32}\ (t\texttt{:valtype})^n & \Rightarrow & t^n \\
\texttt{functype} & ::= & \texttt{0x60}\ t_1^*\texttt{:resulttype}\ t_2^*\texttt{:resulttype} & \Rightarrow & t_1^* \to t_2^*
\end{array}
$$

Finally, instruction opcodes:

$$
\begin{array}{rcllll}
\texttt{globalidx} & ::= & x\texttt{:u32} & \Rightarrow & x \\
\texttt{localidx} & ::= & x\texttt{:u32} & \Rightarrow & x \\
\texttt{instr} & ::= & \texttt{0x01} & \Rightarrow & \mathsf{nop} \\
 & \mid & \texttt{0x1A} & \Rightarrow & \mathsf{drop} \\
 & \mid & \texttt{0x1B} & \Rightarrow & \mathsf{select} \\
 & \mid & \texttt{0x20}\ x\texttt{:localidx} & \Rightarrow & \mathsf{local.get}\ x \\
 & \mid & \texttt{0x21}\ x\texttt{:localidx} & \Rightarrow & \mathsf{local.set}\ x \\
 & \mid & \texttt{0x23}\ x\texttt{:globalidx} & \Rightarrow & \mathsf{global.get}\ x \\
 & \mid & \texttt{0x24}\ x\texttt{:globalidx} & \Rightarrow & \mathsf{global.set}\ x \\
 & \mid & \texttt{0x41}\ n\texttt{:u32} & \Rightarrow & \mathsf{i32.const}\ n \\
 & \mid & \texttt{0x42}\ n\texttt{:u64} & \Rightarrow & \mathsf{i64.const}\ n \\
 & \mid & \texttt{0x43}\ p\texttt{:f32} & \Rightarrow & \mathsf{f32.const}\ p \\
 & \mid & \texttt{0x44}\ p\texttt{:f64} & \Rightarrow & \mathsf{f64.const}\ p
\end{array}
$$